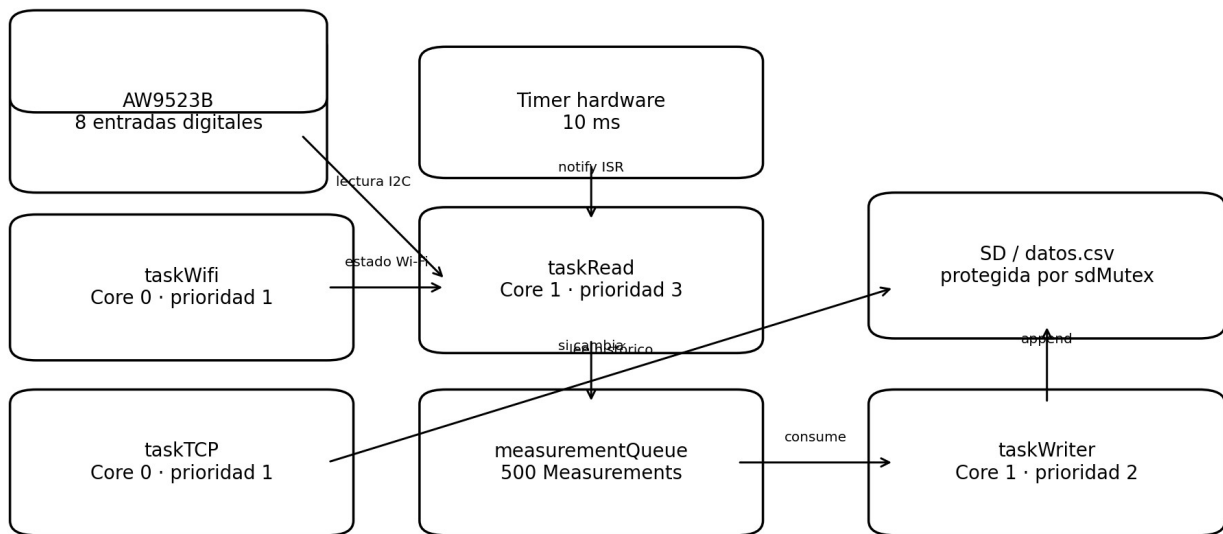


DOCUMENTACIÓN TÉCNICA

StampPLC - adquisición digital, almacenamiento SD y comunicación TCP

Firmware para M5Stack StampPLC / ESP32-S3

Arquitectura funcional del PLC



Separación intencionada: adquisición/escritura en Core 1; comunicaciones en Core 0.

Arquitectura funcional implementada en el firmware actual.

Versión del documento: 17 de septiembre de 2026

Estado: documentación del código actual + decisiones de arquitectura desarrolladas durante el proyecto.

Objetivo

Este documento explica qué se ha construido, cómo se inicializa el PLC, cómo se adquieren las ocho entradas digitales, cómo se desacopla la adquisición mediante una cola en RAM, cómo se almacenan los cambios en la tarjeta SD y cómo un servidor externo puede solicitar el histórico a través de TCP.

Contenido

1. Resumen del sistema
2. Evolución del proyecto
3. Hardware y software
4. Arquitectura general
5. Modelo de datos y lectura de entradas
6. Temporizador y taskRead
7. Cola RAM y taskWriter
8. Tarjeta SD y exclusión mutua
9. Wi-Fi
10. TCP y protocolo de comandos
11. Transferencia de histórico
12. Secuencia de arranque / main.cpp
13. Distribución FreeRTOS y prioridades
14. Estructura de archivos
15. Comportamiento completo extremo a extremo
16. Casos límite y puntos a mejorar
17. Plan de pruebas
18. Referencia rápida

1. Resumen del sistema

El PLC está diseñado como un registrador de cambios de ocho entradas digitales con persistencia local y acceso remoto. La adquisición no transmite directamente por red: primero detecta cambios, los encola en RAM y una tarea independiente los persiste en la SD. La comunicación TCP puede leer ese histórico sin bloquear permanentemente la escritura porque ambas rutas comparten un mutex de SD.

Bloque	Responsabilidad
AW9523B / customMeas	Leer las ocho entradas y convertirlas a un uint8_t.
Timer hardware	Generar el ritmo de muestreo: 10 ms en la configuración actual.
taskRead	Despertar por notificación del timer, detectar cambios y producir Measurement.

Bloque	Responsabilidad
measurementQueue	Buffer RAM entre adquisición y escritura.
taskWriter	Consumir la cola y añadir registros a /datos.csv.
sdMutex	Evitar acceso simultáneo a la SD desde Writer y TCP.
taskWifi	Mantener la conectividad Wi-Fi.
taskTCP	Mantener la conexión TCP, recibir comandos y servir histórico.
Servidor externo	Enviar PING o GET_FROM,<timestamp> y recibir datos.

2. Evolución del proyecto

La implementación actual es el resultado de varias decisiones sucesivas tomadas durante el desarrollo. Esta sección resume esa evolución para conservar el contexto técnico.

- Se comenzó estudiando una implementación con ESP-IDF puro y el driver del expansor AW9523B.
- Posteriormente se migró el firmware a Arduino sobre PlatformIO para simplificar la integración con M5Stack PLC, Wi-Fi, SD y FreeRTOS.
- La lectura de entradas se implementó accediendo al expansor B del StampPLC y leyendo directamente sus registros 0x00 y 0x01.
- La comunicación de red se probó inicialmente con UDP y después se trasladó a TCP para tener conexión, detección de desconexiones y un protocolo de petición/respuesta.
- Se separó el sistema en tareas FreeRTOS: adquisición, escritura, Wi-Fi y TCP.
- Se añadió una cola para evitar que la adquisición dependa de la latencia de la SD.
- Se añadió sdMutex para serializar el acceso a datos.csv entre escritura y lectura histórica.
- Finalmente se implementó el comando GET_FROM para recuperar desde SD todas las muestras con timestamp igual o posterior al solicitado.

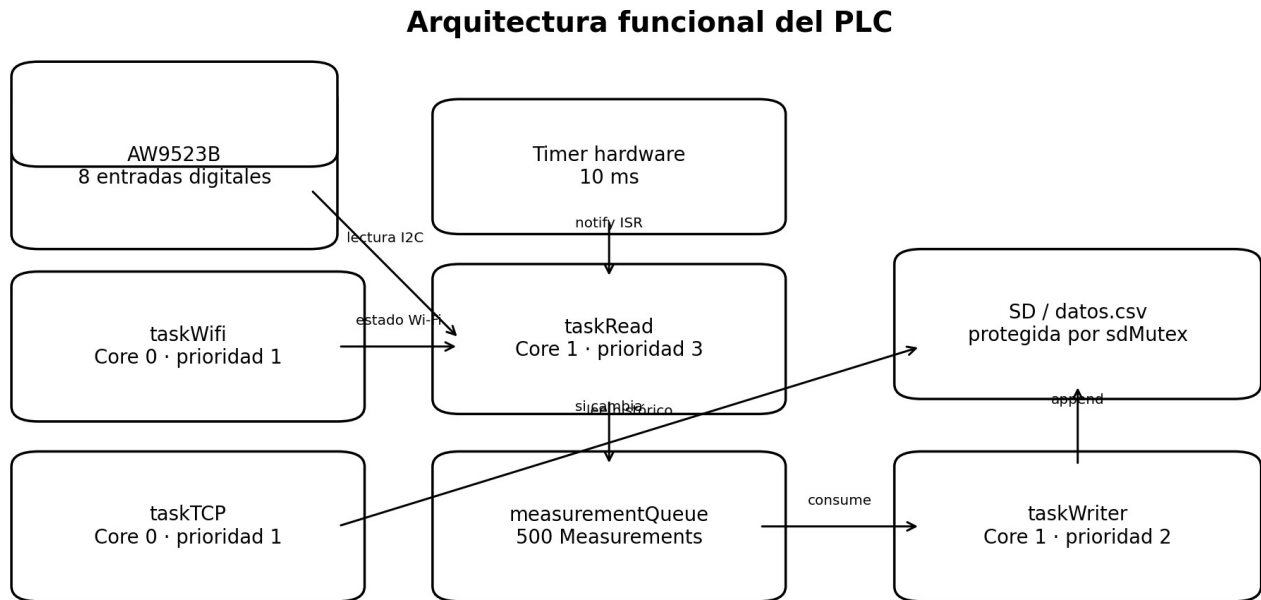
La arquitectura resultante prioriza no perder la cadencia de adquisición por culpa de la red o de la tarjeta SD. La separación no elimina todos los posibles desbordamientos, pero desacopla claramente las responsabilidades.

3. Hardware y software

Elemento	Uso en el proyecto
M5Stack StampPLC / StampS3	Plataforma del PLC basada en ESP32-S3.
AW9523B	Expansor de E/S utilizado por el StampPLC para las entradas digitales.
8 entradas digitales	Mapeadas mediante IN_PIN_LIST = {4,5,6,7,12,13,14,15}.
Tarjeta SD	Persistencia del histórico en /datos.csv.
Wi-Fi STA	Conexión del PLC a una red existente.
TCP	Canal fiable con el servidor externo en el puerto configurado.
Arduino + PlatformIO	Framework y entorno de compilación empleados actualmente.
FreeRTOS	Planificación de tareas, cola, notificaciones y mutex.

Las credenciales Wi-Fi y la IP/puerto del servidor se guardan actualmente en una estructura WifiSetup configurada al arrancar.

4. Arquitectura general



Separación intencionada: adquisición/escritura en Core 1; comunicaciones en Core 0.

Figura 1. Relación entre adquisición, persistencia y comunicaciones.

La separación por tareas tiene dos objetivos principales: mantener la adquisición independiente de operaciones lentas y evitar que las comunicaciones de red interfieran con el almacenamiento. taskRead y taskWriter se ejecutan en el Core 1; taskWifi y taskTCP, en el Core 0.

La adquisición tiene prioridad mayor que la escritura, y la escritura mayor que las comunicaciones. De este modo, una transferencia TCP larga no debería desplazar a taskRead.

5. Modelo de datos y lectura de entradas

La unidad básica de información es Measurement, definida con un timestamp de 32 bits y un byte para el estado de las ocho entradas:

```
struct Measurement {
    uint32_t timestamp;
    uint8_t pinState;
};
```

Cada bit de pinState corresponde a una entrada: bit 0 -> IN1, bit 1 -> IN2, ..., bit 7 -> IN8. Por tanto, pinState puede tomar valores de 0 a 255.

La función leerPines() obtiene el expansor B de M5StamPLC, lee los dos registros de entrada y reconstruye un valor raw de 16 bits:

```
uint16_t raw =
    ((uint16_t)aw.readRegister8(0x01) << 8) |
    (uint16_t)aw.readRegister8(0x00);
```

Después recorre IN_PIN_LIST y copia cada bit físico al bit lógico correspondiente de pinState. La lista utilizada actualmente es:

```
constexpr uint8_t IN_PIN_LIST[8] = {4, 5, 6, 7, 12, 13, 14, 15};
```

lecturaEntrada() sólo genera una medida nueva cuando detecta que el byte de entradas ha cambiado. En ese momento actualiza pinState, toma millis() como timestamp y devuelve true.

Consecuencia importante: el firmware registra transiciones, no una muestra periódica de estado. Si el estado inicial coincide con 0x00, la implementación actual no almacena una primera fila hasta que ocurra el primer cambio.

6. Temporizador y taskRead

El timer hardware se configura para generar un evento cada 10 000 microsegundos, es decir, cada 10 ms. La ISR no lee I2C ni escribe en la SD. Su única responsabilidad es notificar a taskRead:

```
void IRAM_ATTR onReadTimer(){
    BaseType_t taskWoken = pdFALSE;
    vTaskNotifyGiveFromISR(taskReadHandle, &taskWoken);
    if (taskWoken == pdTRUE){ portYIELD_FROM_ISR(); }
}
```

Esta decisión mantiene la interrupción corta. La operación relativamente costosa de lectura del AW9523B se realiza en contexto de tarea.

taskRead permanece bloqueada mediante ulTaskNotifyTake() y se despierta cuando llega la notificación. Si lecturaEntrada() detecta un cambio, intenta insertar Measurement en measurementQueue sin esperar:

```
if (lecturaEntrada(meas)) {
    xQueueSend(measurementQueue, &meas, 0);
}
```

La cola se crea con capacidad para 500 estructuras Measurement. Con la estructura actual, el firmware no comprueba el valor devuelto por xQueueSend(); si la cola se llenase, una transición podría descartarse silenciosamente. Esto se documenta como uno de los puntos de mejora.

7. Cola RAM y taskWriter

measurementQueue desacopla dos velocidades diferentes: la lectura de entradas y la escritura física en la SD. taskRead produce eventos; taskWriter los consume.

taskWriter espera de forma bloqueante una medida. Cuando recibe una, toma sdMutex, escribe una fila en datos.csv y libera el mutex:

```
xQueueReceive(measurementQueue, &meas, portMAX_DELAY);
xSemaphoreTake(sdMutex, portMAX_DELAY);
```

```
bool flag = escribir_dato_a_SD(meas);
xSemaphoreGive(sdMutex);
```

El formato de cada fila es:

```
<timestamp_ms>,<pinState_decimal>
```

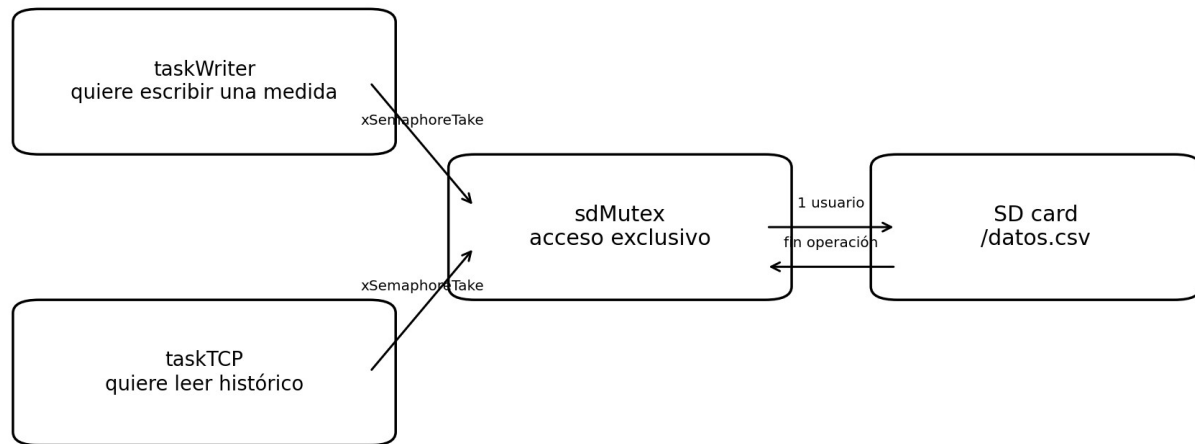
Ejemplo:

```
125430,37
```

El valor 37 en decimal equivale a 0b00100101. La interpretación bit a bit depende de las ocho entradas IN1..IN8 descritas anteriormente.

8. Tarjeta SD y exclusión mutua

Exclusión mutua de la tarjeta SD



Mientras TCP lee un bloque, taskWriter espera. Mientras Writer escribe, TCP espera. La cola RAM desacopla temporalmente la adquisición de la escritura.

Figura 2. Coordinación entre escritura y lectura mediante sdMutex.

customSD implementa tres operaciones: añadir una Measurement, obtener el tamaño actual del archivo y leer un bloque a partir de un offset. Cada operación abre y cierra /datos.csv.

Función	Comportamiento
escribir_dato_a_SD	Abre /datos.csv con FILE_APPEND, escribe timestamp,pinState y cierra.
obtener_size_datos	Devuelve 0 si no existe el archivo; si existe, devuelve file.size().
leer_bloque_datos	Abre el archivo, hace seek(offset), lee hasta maxBytes y cierra.

sdMutex se crea en initTaskWriter(). taskTCP utiliza el mismo mutex cuando necesita inspeccionar o leer la SD. Por eso el orden de inicialización importa: el writer debe inicializarse antes del task TCP.

La lectura histórica toma el mutex sólo durante cada bloque de SD, no durante toda la transmisión. Esto permite que taskWriter intercale escrituras entre bloques y reduce el tiempo continuo de bloqueo.

9. Wi-Fi

WifiSetup agrupa la configuración de red y el objeto WiFiClient:

```
struct WifiSetup {
    IPAddress servidor;
    uint16_t puerto;
    WiFiClient client;
    const char* ssid;
    const char* password;
};
```

taskWifi corre indefinidamente y comprueba WiFi.status(). Cuando la conexión se pierde, ejecuta WiFi.disconnect(), vuelve a lanzar WiFi.begin() y espera hasta recuperar WL_CONNECTED. Después imprime la IP local y vuelve a una comprobación periódica cada 2 s.

La estructura WifiSetup debe vivir durante toda la ejecución porque taskWifi y taskTCP reciben un puntero a ella. Por este motivo, en el main recomendado configWifi se declara con duración global y no como variable automática local de setup().

10. TCP y protocolo de comandos

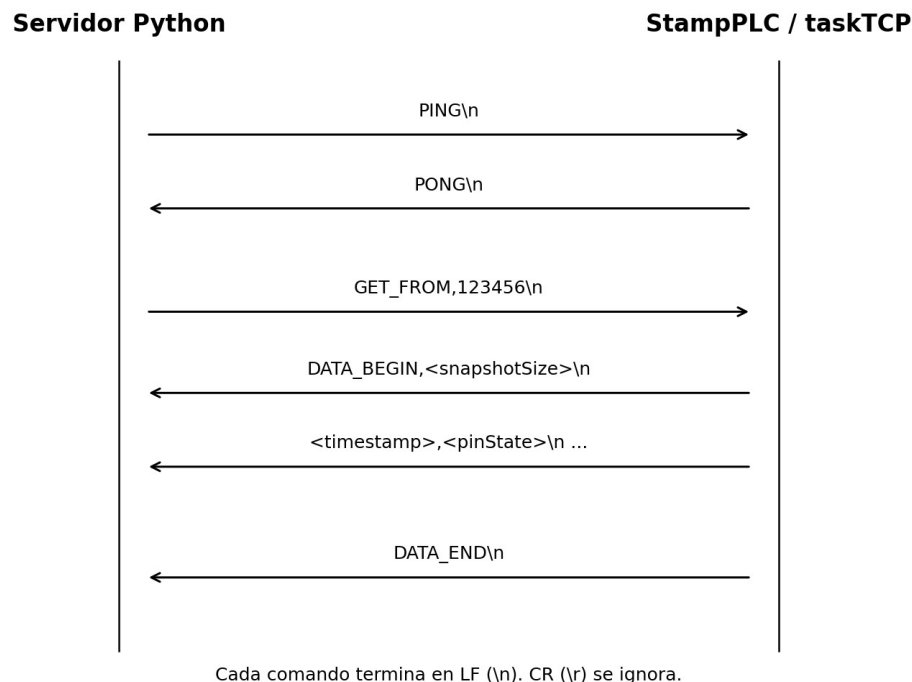


Figura 3. Secuencia de mensajes soportada actualmente.

taskTCP espera a que exista Wi-Fi, asegura una conexión TCP con el servidor y después procesa bytes entrantes. Si la conexión se pierde, limpia el buffer de comando y vuelve a intentar conectar.

Los comandos se reciben como texto ASCII terminado en LF. CR se ignora, por lo que se aceptan terminadores CRLF. El buffer máximo de comando es de 64 bytes.

Comando	Respuesta / efecto
PING	Responde PONG.
GET_FROM,<timestamp>	Envía el histórico con timestamp >= al valor solicitado.
línea vacía	Se ignora.
comando desconocido	ERR,UNKNOWN_COMMAND
timestamp ausente o inválido	ERR,INVALID_TIMESTAMP
comando demasiado largo	ERR,COMMAND_TOO_LONG

El parser usa strtoul() y exige que, tras el número del timestamp, no quede ningún carácter adicional. Esto evita aceptar cadenas parcialmente válidas.

11. Transferencia de histórico

enviar_historico() implementa una transferencia por snapshot. Primero toma sdMutex y captura el tamaño actual del archivo. Ese valor fija el final lógico de la transferencia: los bytes añadidos posteriormente por taskWriter no forman parte de esa petición.

1. Tomar sdMutex y obtener snapshotSize = tamaño actual de datos.csv.
2. Liberar el mutex y enviar DATA_BEGIN,<snapshotSize>.
3. Leer el archivo por bloques de hasta 512 bytes, tomando y liberando sdMutex alrededor de cada lectura.
4. Reconstruir líneas completas en un buffer de 64 bytes.
5. Extraer el timestamp de cada línea y enviar sólo las filas cuyo timestamp sea >= al solicitado.
6. Enviar DATA_END al finalizar.

Este diseño permite que la adquisición y el writer sigan trabajando durante una transferencia extensa. La SD no es utilizada simultáneamente por dos tareas, pero el mutex se libera entre bloques.

Detalle de protocolo importante: el número incluido en DATA_BEGIN es actualmente el tamaño en bytes del snapshot completo del archivo, no el número de filas que se enviarán ni el número de bytes posteriores al filtro de timestamp. El servidor no debe interpretarlo como longitud exacta de la carga filtrada salvo que el protocolo se modifique.

Si una lectura de SD devuelve 0 bytes inesperadamente, la función termina con false. Si el TCP sigue conectado, procesarComandoTCP responde ERR,HISTORY_TRANSFER.

12. Secuencia de arranque / main.cpp

El main debe limitarse a preparar la plataforma, construir la configuración compartida y arrancar las tareas en un orden seguro. La secuencia recomendada es la siguiente:

1. Serial.begin().
2. Habilitar SD en la configuración de M5StamPLC e inicializar la plataforma.
3. Configurar WifiSetup como objeto global/persistente.
4. initTaskRead(): crea measurementQueue y taskRead.
5. initTaskWriter(): crea sdMutex y taskWriter.
6. initTaskWifi(configWifi).
7. initTaskTCP(configWifi).
8. initReadTimer() al final, cuando taskReadHandle ya existe y el resto del pipeline está preparado.

Esqueleto recomendado:

```
#include <Arduino.h>
#include <M5StamPLC.h>
#include <customWifi.h>
#include <customTask.h>

WifiSetup configWifi;

void setup() {
    Serial.begin(115200);

    auto config = M5StamPLC.config();
    config.enableSdCard = true;
    M5StamPLC.config(config);
    M5StamPLC.begin();

    configWifi.ssid = "test_server_plc";
    configWifi.password = "<password>";
    configWifi.servidor = IPAddress(192, 168, 50, 1);
    configWifi.puerto = 5000;

    initTaskRead();
    initTaskWriter();
    initTaskWifi(configWifi);
    initTaskTCP(configWifi);
    initReadTimer();
}

void loop() {
    vTaskDelay(pdMS_TO_TICKS(1000));
}
```

El timer debe arrancar al final porque, desde el momento en que se habilita, puede generar notificaciones hacia taskRead.

13. Distribución FreeRTOS y prioridades

Tarea	Core	Prioridad	Stack	Bloqueo principal
taskRead	1	3	4096	ulTaskNotifyTake()
taskWriter	1	2	4096	xQueueReceive() / sdMutex
taskWifi	0	1	4096	vTaskDelay() / espera Wi-Fi
taskTCP	0	1	6144	vTaskDelay() / lectura TCP / sdMutex

La prioridad refleja la criticidad temporal: leer primero, persistir después y comunicar en último lugar. taskTCP dispone de más stack porque mantiene buffers y ejecuta la lógica de transferencia histórica.

Los mecanismos de sincronización empleados son distintos porque resuelven problemas distintos: notificación directa para timer -> taskRead, cola para transferir Measurements entre tareas, y mutex para proteger un recurso compartido no reentrante: la SD.

14. Estructura de archivos

Archivo	Responsabilidad
main.cpp	Configuración global e inicialización del sistema.
customMeas.h/cpp	Measurement, lectura del AW9523B y detección de cambios.
customSD.h/cpp	Escritura y lectura por bloques de /datos.csv.
customWifi.h/cpp	WifiSetup, conexión Wi-Fi auxiliar y conexión TCP básica.
customTCP.h/cpp	Parser del protocolo, reconexión TCP y envío de histórico.
customTask.h/cpp	Timer, objetos FreeRTOS y las cuatro tareas principales.

Esta separación mantiene la lógica de hardware, almacenamiento, comunicaciones y scheduling en módulos diferentes. main.cpp no necesita conocer los detalles internos de cada tarea.

15. Comportamiento completo extremo a extremo

Ejemplo de una transición real de entrada:

1. El timer alcanza 10 ms y ejecuta onReadTimer().
2. La ISR incrementa la notificación de taskRead y sale inmediatamente.
3. taskRead lee los registros del AW9523B y reconstruye las ocho entradas.
4. Si el byte difiere del último pinState, crea/actualiza Measurement con millis().
5. Measurement entra en measurementQueue.

6. taskWriter despierta, toma sdMutex y añade la fila a /datos.csv.
7. Más tarde el servidor envía GET_FROM,<timestamp>.
8. taskTCP toma un snapshot del tamaño del archivo y lo lee por bloques, compartiendo la SD con taskWriter mediante sdMutex.
9. Las filas con timestamp suficiente se envían por TCP y la transferencia termina con DATA_END.

Si durante la transferencia aparece un nuevo cambio de entrada, taskRead puede seguir encolándolo. taskWriter puede escribirlo cuando obtenga el mutex. Como el snapshot ya fue capturado, ese nuevo registro quedará para una petición posterior.

16. Casos límite y puntos a mejorar

Tema	Estado actual / riesgo	Mejora recomendada
Cola llena	xQueueSend(...,0) no se comprueba; un evento puede perderse.	Comprobar retorno, contador de pérdidas o política de backpressure.
Estado inicial	Si comienza en 0x00 no se escribe hasta el primer cambio.	Registrar una muestra inicial explícita en el arranque.
millis()	uint32_t se desborda aproximadamente cada 49,7 días.	Gestionar rollover o usar contador/epoch de 64 bits.
DATA_BEGIN	snapshotSize es tamaño bruto del archivo, no payload filtrado.	Documentar formalmente o cambiar el campo a filas/bytes enviados.
Credenciales	SSID/password están en firmware.	Mover a configuración segura/provisionado si el producto lo requiere.
SD	Cada bloque abre/cierra el archivo.	Valorar handle persistente si el rendimiento lo exige.
Errores de init	Las funciones imprimen errores, pero setup puede continuar.	Propagar bool/estado y abortar o entrar en modo seguro.
Protocolo	Sin checksum, versión, autenticación ni ACK por lote.	Añadir sólo si los requisitos de robustez/seguridad lo justifican.
Integridad al apagar	No existe procedimiento explícito de shutdown.	Evaluar flush/estado de alimentación si hay cortes frecuentes.

Estos puntos no invalidan la arquitectura. Son aspectos que conviene cerrar antes de considerar el firmware terminado o desplegarlo de forma continua.

17. Plan de pruebas

Para validar el sistema completo conviene realizar las siguientes pruebas funcionales:

- Arranque sin servidor TCP disponible: Wi-Fi debe permanecer operativo y taskTCP debe reintentar sin afectar a la adquisición.
- Desconectar y reconectar Wi-Fi: taskWifi debe recuperar WL_CONNECTED y taskTCP debe abrir un nuevo socket.

- Reiniciar el PLC con el servidor activo: el servidor debe aceptar la nueva conexión sin requerir reinicio manual.
- PING: comprobar PING -> PONG repetidamente.
- Cambiar cada una de las ocho entradas y verificar el bit correspondiente en datos.csv.
- Provocar cambios rápidos mientras se ejecuta GET_FROM y comprobar que no hay corrupción del CSV.
- Solicitar un timestamp antiguo, reciente, futuro y exactamente igual al de una muestra.
- Enviar comando vacío, desconocido, timestamp no numérico y comando de más de 63 caracteres.
- Retirar/fallar la SD y verificar los mensajes ERR,SD / ERR,HISTORY_TRANSFER según el punto de fallo.
- Forzar una cola saturada para comprobar la política futura de detección de pérdidas.

Una prueba de carga especialmente útil es generar cambios continuos durante una transferencia de histórico larga y comparar el CSV final con los eventos recibidos por el servidor.

18. Referencia rápida

Parámetro	Valor actual
Periodo de lectura	10 ms
Número de entradas	8
Tipo de timestamp	uint32_t, millis()
Formato de pinState	uint8_t; guardado decimal en CSV
Capacidad measurementQueue	500 elementos
Archivo histórico	/datos.csv
Buffer comando TCP	64 bytes
Bloque lectura SD	512 bytes
Buffer línea SD	64 bytes
Comandos TCP	PING; GET_FROM,<timestamp>
Terminador de comando	LF; CR ignorado
Puerto de ejemplo	5000
Core adquisición/escritura	Core 1
Core comunicaciones	Core 0

Mensajes de protocolo principales:

```

PING
PONG
GET_FROM,<timestamp>
DATA_BEGIN,<snapshotSize>
<timestamp>,<pinState>
...
DATA_END
ERR,INVALID_TIMESTAMP
ERR,COMMAND_TOO_LONG
ERR,UNKNOWN_COMMAND
ERR,SD
ERR,HISTORY_TRANSFER

```

Fuentes revisadas para esta versión

Código actual aportado en el proyecto: customTCP.cpp/.h, customWifi.cpp/.h, customMeas.cpp/.h, customSD.cpp/.h, customTask.cpp/.h y main.cpp. La sección de main recomendado incorpora la secuencia de inicialización definida inmediatamente antes de solicitar esta documentación.